

HiDiTingV100 NV 存储

用户指南

文档版本 00B01

发布日期 2025-06-05

前言

概述

本文档主要针对 HiDiTingV100 中 NV 存储模块的使用进行介绍。用于指导工程人员能够快速使用 NV 模块进行二次开发。

产品版本

与本文档相对应的产品版本如下。

产品名称	产品版本
HiDiTing	V100

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 软件工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
----	----

符号	说明
 危险	表示如不避免则将会导致死亡或严重伤害的具有高等级风险的危害。
 警告	表示如不避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不避免则可能导致轻微或中度伤害的具有低等级风险的危害。
 须知	用于传递设备或环境安全警示信息。如不避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
00B01	2025-06-05	第一次临时版本发布。

目 录

前言.....i

1 NV 简介.....1

2 NV 编译预置.....2

2.1 新增 NV 项 2

2.1.1 新增 NV 项流程 2

2.1.2 新增 NV 项示例 4

2.2 编译生成 NV 镜像..... 6

3 NV API 指南.....7

3.1 功能描述..... 7

3.2 接口说明..... 8

3.3 开发指引..... 10

3.4 注意事项..... 12

3.5 常见问题..... 12

插图目录

图 2-1 NV 配置文件示例 3

图 2-2 添加 kvalue 预置值为基础类型..... 5

图 2-3 添加 kvalue 预置值为结构体类型 6

表格目录

表 2-1 NV 配置选项说明 3

Draft

1 NV 简介

NV 模块用于本地存储器中存储非易失性数据。NV 中的每项数据以类似 key-value 的方式进行定义，数据项中包含唯一的索引 key 和自定义数据类型的 value。

NV 项可通过两种方式进行存储：编译预置和 API 写入。

- 编译预置是指开发者可在代码编译阶段，通过修改 NV 头文件和 NV 配置文件的方式生成定制化的 NV 镜像，在镜像烧录的过程中统一烧录到存储介质中。预置的 NV 在代码运行阶段可通过 API 接口进行读取和更新。
- API 写入是用户可直接在代码中调用 API 接口写入新的 NV 项，具体使用方法请参见“[3 NV API 指南](#)”章节。

以 Flash 为例，NV 项存储在 Flash 中时，以 Flash 器件的 sector 为单位进行管理，NV 的每个页为一个 sector 大小（4096Byte），NV 页的数量默认配置为 3 页。除去管理结构，单个非加密 NV 项的有效数据最大不应超过 4060Byte。

2 NV 编译预置

2.1 新增 NV 项

2.2 编译生成 NV 镜像

2.1 新增 NV 项

2.1.1 新增 NV 项流程

新增 NV 项流程如下：

步骤 1 在头文件中新增 kvalue 的数据类型定义（非必须，如果是通用类型数据可忽略此步骤）。

步骤 2 在 json 文件中新增 NV 描述项。

-----结束

新增 kvalue 数据类型

- 通用数据类型：uint8_t、uint16_t、uint32_t、bool。
- 自定义数据类型：支持自定义枚举（enum）类型和结构体（struct）类型。
- 自定义数据类型存放路径：
middleware/chips/brandy/nv/nv_config/include/common.h
- 当用户使用通用或已定义的数据类型时，不涉及本小节的修改；当用户要新增枚举或结构体类型时，需在上述文件中定义。

新增 NV 描述项

- NV 描述项文件路径：middleware/chips/brandy/nv/nv_config/cfg/acore/app.json
- 定义说明：

表2-1 NV 配置选项说明

NV 配置选项	说明
key_id	NV 项的 ID。
key_status	NV 项的状态。
structure_type	NV 项的数据结构类型。
attributions	NV 项的属性值。
value	NV 项的数据。

图2-1 NV 配置文件示例

```
"common":{
  "module_id": "0x0",

  "unused":{
    "key_id": "0x0",
    "key_status": "reserve",
    "structure_type": "uint8_t",
    "attributions": 1,
    "value": [0]
  },
  "sample":{
    "key_id": "0x1",
    "key_status": "alive",
    "structure_type": "sample_type_t",
    "attributions": 1,
    "value": [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17]
  }
}
```

在 NV 的配置中，每个字段的详细描述如下：

- key_id:
以十六进制形式给出的 NV 项 ID。key_id 必须唯一，不能重复，因此建议用户对 key_id 的取值进行管理，如：key_id 由 16bit 组成，可规定高 8bit 为所

在模块内的 module_id，低 8bit 在模块内取值，避免不同模块使用 NV 互相影响。

- key_status:

用于标记是否将该项的 NV 值编到生成的 bin 文件中。该字段为 “alive”，表示 NV 项生效，当前固件版本正在使用此 key；若为其他的字段或空，则不生效。

- structure_type:

NV 项的数据类型。已在“[新增 kvalue 数据类型](#)”中详细描述。

- attributions: NV 项属性值。1、2、4 为互斥关系，三选一。

1: Normal nv (普通 NV)。

2: Permanent nv (不可修改)。

4: Un-upgrade nv (不随版本升级而修改)。

- value:

如果 value 不是上述通用数据类型，任何结构都必须以列表的形式书写，有如下两种情况：

- 列表所有成员全部赋值。
- 只对列表前面若干个成员赋值。这表明对末尾未赋值的成员缺省赋值为 0。

2.1.2 新增 NV 项示例

- 在 “middleware/chips/brandy/nv/nv_config/include/common.h” 文件中新增自定义结构体。新增自定义数据类型示例如下：

- 新增结构体类型且结构体内都为基础类型：

```
typedef struct {  
    int8_t param1;  
    int8_t param2;  
    int8_t param3;  
    int8_t param4;  
    int8_t param5;  
    uint32_t param6;  
    uint32_t param7;  
    int32_t param8;  
    uint32_t param9;  
    uint32_t param10;  
    uint32_t param11;  
}
```

```
uint32_t param12;
uint32_t param13;
uint32_t param14;
uint32_t param15;
uint32_t param16;
uint32_t param17;
} sample_type_t;
```

- 新增结构体类型且结构体内有数组类型:

```
typedef struct {
    uint16_t param1;
    uint16_t param2;
    uint16_t param3;
    uint16_t param4[2];
} sample_two;
```

- 新增枚举类型:

```
typedef enum {
    PARAM1,
    PARAM2,
    PARAM3,
    PARAM4
} sample_three;
```

- 在 “middleware\chips\brandy\nv\nv_config\cfg\core\app.json” 文件中添加新的 NV 项。
 - 当 kvalue 预置值类型是基础类型时，添加 kvalue 预置值如图 2-2 所示，可在 “app.json” 配置文件直接添加，无需在头文件中新增。

图2-2 添加 kvalue 预置值为基础类型

```
"sample1":{
    ... "key_id": "0x1",
    ... "key_status": "alive",
    ... "structure_type": "uint8_t",
    ... "attributions": 1,
    ... "value": 0
},
```

- 当 kvalue 预置值类型是结构体类型时，添加 kvalue 预置值如图 2-3 所示，该 kvalue 值要对应头文件中已有的结构体，如果没有需手动添加自定义结构体。

图2-3 添加 kvalue 预置值为结构体类型

```
"sample2":{
  ... "key_id": "0x2",
  ... "key_status": "alive",
  ... "structure_type": "sample_type_t",
  ... "attributions": 1,
  ... "value": [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17]
}
```

说明

当添加了结构体并如图 2-3 调用后，需先编译任意 A 核版本生成必要中间文件后再执行“2.2 编译生成 NV 镜像”中编译命令，才能生成 NV 镜像。

2.2 编译生成 NV 镜像

重新执行工程编译命令，将会自动生成 NV 镜像，并且打包到用于烧写的 fwpkg 文件中。

3 NV API 指南

3.1 功能描述

3.2 接口说明

3.3 开发指引

3.4 注意事项

3.5 常见问题

3.1 功能描述

NV API 主要提供以下几种功能：

- NV 项写入：

保存需要存储的格式化数据，同时可以定义写入数据的三种属性，分别为是否永久存储，是否加密存储和是否不可升级。

- NV 项读取：

从本地存储器读取 NV 数据。

- NV 信息查询：

- 查询 NV 是否已存储于本地存储器中。
- 查询 NV 空间的使用状态。

- NV 刷新缓存中数据到 flash：

仅 NV 支持异步存储时有效，在下电、关机、重启等场景调用，保证 NV 数据不丢失。

NV 项的写入接口可设置 NV 项的属性，对于通过 API 动态添加的 NV 项，可在其写入的接口中传入其所拥有的特殊属性。

3.2 接口说明

使用 NV 接口需要引用 NV 接口头文件，路径：include/middleware/utls/nv.h

NV 模块主要提供以下 API：

errcode_t uapi_nv_write(uint16_t key, const uint8_t *kvalue, uint16_t kvalue_length)

uapi_nv_write	写入 NV 数据项，默认属性 Normal，无回调函数。
key	要写入的 NV 项的 key ID，用于索引。
*kvalue	指向要写入的 NV 项的值的指针。
kvalue_length	写入数据的长度，单位：Byte。

errcode_t uapi_nv_write_with_attr(uint16_t key, const uint8_t *kvalue, uint16_t kvalue_length, nv_key_attr_t *attr, nv_storage_completed_callback func)

uapi_nv_write_with_attr	写入 NV 数据项，并根据业务需求配置属性及回调函数。
key	要写入的 NV 项的 key ID，用于索引。
*kvalue	指向要写入的 NV 项的值的指针。
kvalue_length	写入数据的长度，单位：Byte。
*attr	要配置的 NV 项的属性。
func	kvalue 写入 Flash 后调用的回调函数。

errcode_t uapi_nv_read(uint16_t key, uint16_t kvalue_max_length, uint16_t *kvalue_length, uint8_t *kvalue)

uapi_nv_read	读取指定 NV 数据项的值，默认不获取 key 的属性值。
key	要读取的 NV 项的 key ID，用于索引。
kvalue_max_length	允许存储数据的最大长度，单位：Byte。

uapi_nv_read	读取指定 NV 数据项的值，默认不获取 key 的属性值。
*kvalue_length	实际读取到的数据长度。
*kvalue	指向保存读取数据的 buffer 的指针。

```
errcode_t uapi_nv_read_with_attr(uint16_t key, uint16_t kvalue_max_length, uint16_t
*kvalue_length, uint8_t *kvalue, nv_key_attr_t *attr)
```

uapi_nv_read_with_attr	读取指定 NV 数据项的值，同时获取 key 的属性值
key	要读取的 NV 项的 key ID，用于索引。
kvalue_max_length	允许存储数据的最大长度，单位：Byte。
*kvalue_length	实际读取到的数据长度。
*kvalue	指向保存读取数据的 buffer 的指针。
*attr	获取到的 NV 项的属性。

```
errcode_t uapi_nv_get_store_status(nv_store_status_t *status)
```

uapi_nv_get_store_status	获取 NV 存储空间使用情况。
*status	指向保存 NV 状态数据的指针。

```
errcode_t errcode_t uapi_nv_flush(void)
```

uapi_nv_flush	NV 支持异步存储时，刷新缓存 NV 数据到 flash。
void	该函数入参为空，下电前调用。

须知

uapi_nv_flush：该接口为 NV 支持异步存储时新增接口，在关机、下电、重启等流程要主动增加该接口的调用，保证数据不丢失。

3.3 开发指引

使用 NV 接口需要引用 NV 接口头文件，路径：include/middleware/utls/nv.h

下述步骤为 NV 读写接口的使用指引。API 调用处以加粗突出。

步骤 1 写入默认 Normal 类型 NV。

```
uint8_t *test_nv_value; /* 要写入的NV value保存在test_nv_value中 */

uint32_t test_len = 15; /* 长度为test_len , 例中为15*/

uint16_t key = TEST_KEY; /* TEST_KEY 为该Key的ID*/

uint16_t key_len= test_len;
uint8_t *write_value = uapi_malloc(key_len);
(void)memcpy_s(write_value, key_len, test_nv_value, key_len);
errcode_t nv_ret_value = uapi_nv_write(key, write_value, key_len);
if (nv_ret_value != ERRCODE_SUCC) {
    /* ERROR PROCESS */
    uapi_free(wrt_value);
    return ERRCODE_FAIL;
}
/* APP PROCESS */
uapi_free(wrt_value);
return ERRCODE_SUCC;
```

步骤 2 写入带属性 NV（配置加密属性，其他略）。

```
uint8_t *test_nv_value; /* 要写入的NV value保存在test_nv_value中 */

uint32_t test_len = 15; /* 长度为test_len, 例中为15 */

uint16_t key = TEST_KEY;
uint16_t key_len= test_len;

nv_key_attr_t attr = {0};
attr.permanent = false;
attr.encrypted = true; /* 加密属性设为true */
attr.non_upgrade = false
uint8_t *write_value= uapi_malloc(key_len);
(void)memcpy_s(write_value, key_len, test_nv_value, key_len);
errcode_t nv_ret_value = uapi_nv_write_with_attr(key, write_value, key_len, &attr, NULL);
```

```
if (nv_ret_value != ERRCODE_SUCC) {  
    /* ERROR PROCESS */  
    uapi_free(write_value);  
    return ERRCODE_FAIL;  
}  
/* APP PROCESS */  
uapi_free(write_value);  
return ERRCODE_SUCC;
```

步骤 3 读取 NV。

```
uint16_t key = TEST_KEY;  
uint16_t key_len = test_len;  
uint16_t real_len = 0;  
uint8_t *read_value = uapi_malloc(key_len);  
if (uapi_nv_read(key, key_len, &real_len, read_value) != ERRCODE_SUCC) {  
    /* ERROR PROCESS */  
    uapi_free(read_value);  
    return ERRCODE_FAIL;  
}  
/* APP PROCESS */  
uapi_free(read_value);  
return ERRCODE_SUCC;
```

步骤 4 读取 NV 及属性。

```
uint16_t key = TEST_KEY;  
uint16_t key_len = test_len;  
uint16_t real_len = 0;  
uint8_t *read_value = uapi_malloc(key_len);  
nv_key_attr_t attr = {false, false, false, 0};  
ext_errno nv_ret = uapi_nv_read_with_attr(key, key_len, &real_len, read_value, &attr);  
if (nv_ret != ERRCODE_SUCC) {  
    /* ERROR PROCESS */  
    uapi_free(read_value);  
    return ERRCODE_FAIL;  
}  
/* APP PROCESS */  
uapi_free(read_value);  
return ERRCODE_SUCC;
```

步骤 5 查询 NV 空间状态。

```
nv_store_status status;  
if (uapi_nv_get_store_status(&status) == ERRCODE_SUCC) {  
    /* APP PROCESS */  
    printf("Total:      %d Bytes\r\n", status.total_space);  
    printf("used:       %d Bytes\r\n", status.used_space);  
    printf("reclaimable: %d Bytes\r\n", status.reclaimable_space);  
    printf("corrupted:  %d Bytes\r\n", status.corrupted_space);  
    printf("max_key:    %d Bytes\r\n", status.max_key_space);  
}
```

----结束

说明

开发指引只是 API 接口的测试用例，为用户提供简单的 sample 参考，sample 中省略了宏、部分变量、回调函数的定义过程和业务处理过程。

3.4 注意事项

- uapi_nv_write：默认不对所存储的 key 添加额外属性（是否永久存储、是否加密存储等）。
- BrandyK100 不支持加密 NV。
- uapi_nv_write_with_attr：可同时配置 key 属性和注册回调函数。在 BrandyK100 中回调函数可以忽略，传 NULL 即可。
- NV 属性结构体和 NV 空间状态结构体说明详见“nv.h”文件。

3.5 常见问题

NV 数据读取失败：

- 请排查 uapi_nv_flush 接口是否未在下电前调用，导致数据丢失。